

Greg's List
Analyse Post-Mortem

Travail présenté à
M Sébastien Rousseau
M Pier-Luc Poulin

Par
Anthony Lacasse

Pour le cours
Projet Pratique II

12 décembre 2024

Collège Bart

1. Description du projet

Le projet avait pour but de réaliser un jeu avec des *gameplay ingredients* ajoutés de façon aléatoire pour donner au joueur une expérience différente à chaque partie. Le joueur est placé dans un contexte de maison close avec une simple liste rédigée lui expliquant des tâches à accomplir, parfois de façon plus ou moins ambiguë, pour survivre à son passage dans l'énigmatique maison.

Lorsque le joueur charge la scène de jeu en appuyant sur le bouton "Star Game" au menu principal, il se retrouve dans le hall de la maison. Toutes les portes et fenêtres menant à l'extérieur sont verrouillées, mais la circulation dans la maison est possible dans toutes les pièces, à l'exception du sous-sol. Le jeu, à proprement parler, débute lorsque le joueur interagit avec la feuille de papier fixée sur le dos de la porte principale. Au moment de la collecte, l'algorithme de sélection choisit au hasard une règle parmi celles qui ont été données au singleton RuleManager pour chacune des phases de jeu. La première règle, qui se veut être plus une règle de tutoriel pour permettre au joueur de se familiariser avec les contrôles et l'environnement, est aussi initialisée au même moment et les actions du joueur sont alors mesurées. Le joueur a droit à trois erreurs (comme au base-ball) avant d'échouer la partie et d'être redirigé vers le menu principal. Les erreurs sont comptabilisées en fonction des actions du joueur et à la complétion de chacun des blocs où une fonction de transition vers le bloc suivant valide si tout ce qui était attendu a été accompli. Si le joueur remplit les attentes et complète les blocs de jeu, il peut alors sortir de la maison et est reconduit au menu principal.

À chaque partie, l'algorithme fait en sorte que les règles à suivre, mais aussi l'emplacement des objets dans le niveau, est différent, ce qui donne une toute nouvelle expérience de jeu et donc permet la rejouabilité.

2. Ce qui a moins bien fonctionné

La création de l'atmosphère a clairement été un défi de taille. La scène de jeu contient une Skybox dynamique qui permet de donner l'impression de cycle jour et nuit. Le singleton RuleManager permet de passer une valeur à la Skybox en fonction des actions posées et de la règle en cours dans le but de changer l'illumination extérieur et venir ajouter un sentiment de progression dans la partie. Cependant, j'ai pris pour acquis qu'avec mon expérience d'éclairage en temps réel dans Unreal Engine, que faire l'éclairage dans Unity serait identique. L'absence de l'outil *Photon Catcher* qui aurait permis de concentrer l'éclairage extérieur à l'intérieur de la maison n'existe pas dans Unity, et les lightmaps dynamiques n'ont pas eu l'effet désiré. J'ai passé énormément de temps à tenter de fixer l'ambiance intérieur au lieu de travailler sur d'autres mécaniques, comme l'audio ou plus de règles individuelles.

Parlant des règles, le développement de la mécanique a aussi été un défi. J'ai découvert sur le tard que les ScriptableObject de Unity ne peuvent pas contenir d'éléments d'une scène puisque l'objet ne peut pas être certain que l'élément sera bien présent dans tous les contextes. J'ai passé beaucoup de temps à expérimenter pour trouver une alternative. J'ai finalement trouvé une solution en retournant au ScriptableObject et en créant un script "générique" de niveau qui détient tout ce qu'un niveau doit contenir, couplé avec le singleton RuleManager pour faire l'intermédiaire

entre les deux. Encore une fois, du temps passé à rechercher et expérimenter pour revenir au plan initial.

Le menu principal a quelques difficultés en fonction de la résolution d'écran. Le background s'affiche correctement, mais j'ai des éléments dynamiques qui ont de la difficulté à s'afficher de manière constante. Ça va nécessiter plus d'attention dans le futur, mais c'est sûrement quelque chose qui peut se régler facilement, mais je n'ai jamais pris le temps de regarder plus en détail ce problème, principalement parce que je ne l'ai jamais observé sur mon poste de travail. En fait, le UI en entier est une zone de vigilance pour le projet. Comme c'est le UI qui permet de communiquer avec le joueur, j'aurais pu trouver une autre méthode pour avoir un seul bloc de texte au lieu de plusieurs préfabs instanciés, vu que Unity ne gère pas des tailles dynamiques distinctes avec un même modèle de préfab. Ça fait partie des apprentissages et c'est un bon point de départ lorsque viendra le temps de planifier l'avenir.

3. Ce qui a bien fonctionné

Le singleton RuleManager a bien fonctionné. Même s'il y a eu des difficultés avec les règles en soi, j'ai beaucoup travaillé sur ce système qui était très rigide et hard codé au début. Avant l'élaboration du script de Niveau, le singleton contenait toute l'information et il n'aurait pas été possible de le réutiliser sans le réécrire complètement. Maintenant, on peut réutiliser ce singleton dans toutes les scènes de jeu sans avoir besoin de faire de changements majeurs. La séparation des différents éléments est définitivement un des gros points forts du projet. En réévaluant le code, j'ai réalisé qu'il a moyen de simplifier encore davantage en retravaillant le couplage et en traitant le singleton comme un service provider.

Je suis aussi très fier de la façon dont j'ai construit les règles, particulièrement de la façon dont je fais leur initialisation pour avoir un côté aléatoire et différent à chaque partie. L'aspect de donner une expérience différente à chaque partie est clairement présent grâce à ce point. J'aurais vraiment aimé pouvoir avoir un peu plus de blocs fonctionnels à montrer, mais le fait d'avoir plusieurs chemins différents avec le même jeu, c'est vraiment plus que ce que je pouvais espérer.

4. Quoi faire différemment dans le futur

Je crois que commencer par faire une bonne analyse de ce qui est possible de faire avec les moyens disponibles serait un bon départ. En cherchant à faire un jeu de style horreur sans avoir accès aux éléments classiques de jeux d'épouvante a été un peu problématique. Si j'avais plutôt abordé le projet comme un jeu de puzzle sans cadenas, j'aurais pu aller plus loin dans le développement du projet.

Avoir su plus tôt ce qu'est un listener/observer, j'aurais pris ce chemin directement. Le singleton RuleManager n'est pas un listener, plutôt un service locator à la rigueur, et je crois qu'il aurait été plus simple de faire du débogage avec le patron qu'avec un level, un singleton, une classe abstraite, un player controller et plusieurs objets circonstanciels.

5. Conclusion

Je suis vraiment très fier de mon projet. Ce n'est pas aussi accompli que je l'aurais souhaité, mais on doit jouer avec les cartes qu'on se fait dealer et 2 semaines de maladie en fin de dernière session, même si on ne souhaite ça à personne, ça arrive. J'ai appris énormément en réalisant ce projet, le feedback que j'ai reçu est très encourageant et me motive à continuer le développement dans le futur proche. J'ai une direction claire sur la documentation à consulter et les connaissances à acquérir pour régler les éléments à adresser rapidement et encore plus d'idées pour rendre l'expérience encore meilleure pour les joueurs.